

系统开发工具基础

第二周实验报告

姓名：Yang Kunwang

学号：25020007154

1 课堂

1.1 q05: 控制一个可清理的后台任务

实验内容

```
mkdir -p q05
创建实验目录 q05
cd q05
切换至 q05 工作目录
vim worker.sh
使用vim创建后台任务脚本
chmod +x worker.sh
赋予脚本可执行权限
./worker.sh > stdout.log 2> stderr.log
前台启动脚本，重定向输出日志
# 按下快捷键 Ctrl+Z
挂起前台运行的任务
bg
将任务放到后台继续运行
jobs
查看后台任务状态
kill $(jobs -p)
获取PID，发送SIGTERM终止进程
wait
等待后台进程完全退出
```

实验结果

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+ 已停止                  ./worker.sh > stdout.log 2> stderr.log
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ jobs
[1]+ 运行中                  ./worker.sh > stdout.log 2> stderr.log &
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ kill $(jobs -p)
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ jobs
[1]+ 已完成                  ./worker.sh > stdout.log 2> stderr.log
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ ps aux | grep worker.sh
ykw      4698  0.0  0.0  9756 2684 pts/0    S+   16:34   0:00 grep --color=auto worker.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ cat clean.log
cat: clean.log: No such file or directory (os error 2)
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$ cat cleanup.log
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q05$
```

截图中 `cat cleanup.log` 的输出包含字符串 `CLEAN_EXIT`，说明脚本成功捕获 `SIGTERM` 信号，预先设置的 `trap` 清理代码已经执行。

执行 `jobs` 命令后后台任务列表为空，证明 `worker.sh` 进程已经退出。

`stdout.log` 内存在连续递增的数字，验证脚本在后台成功恢复运行。

`stderr.log` 无报错输出，脚本运行全程无异常，所有实验目标均已达成。

1.2 q06: 语义重构与本地开发反馈

实验内容

```
mkdir -p q06
```

创建实验目录 `q06`

```
cd q06
```

切换工作目录至 `q06`

```
vim math_utils.py
```

创建 `math_utils.py` 并写入函数定义代码

```
vim app.py
```

创建 `app.py` 并写入调用代码

```
vim test_math_utils.py
```

创建 `test_math_utils.py` 测试文件

```
code .
```

使用 VS Code 打开当前文件夹，启动 Python 语言服务器

在 VS Code 图形界面操作

1. 右键 `total_price`，执行转到定义、查找所有引用

2. 使用重命名符号，将 `total_price` 修改为 `calculate_total`

3. 在 `app.py` 添加无用导入 `import os`，由 `ruff` 检测并移除

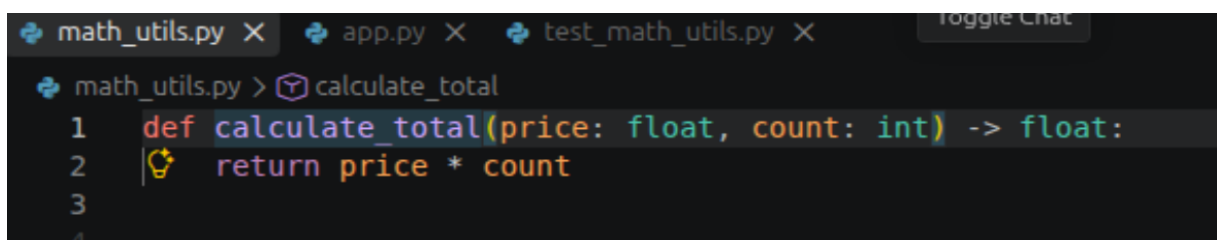
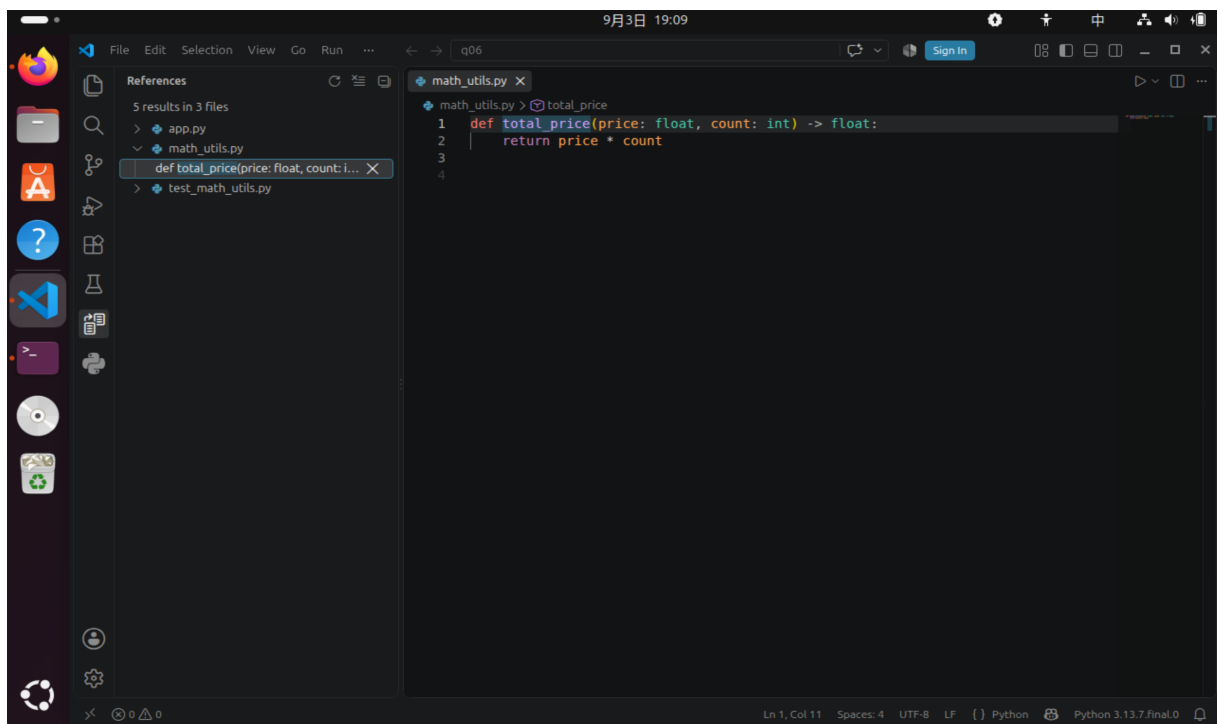
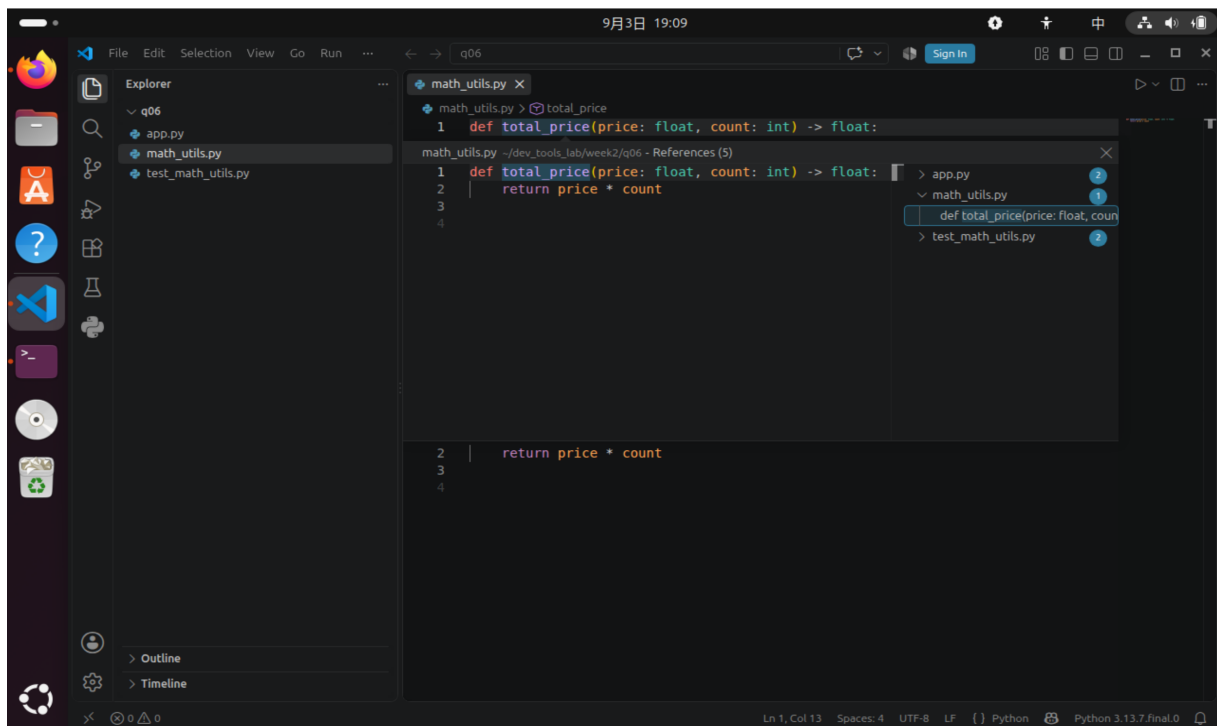
```
ruff check .
```

`ruff` 工具检查项目代码规范

```
pytest test_math_utils.py -v
```

运行 `pytest` 单元测试，验证重构后代码正确性

实验结果



```
math_utils.py X app.py X test_math_utils.py X
app.py
1 from math_utils import calculate_total
2 print(calculate_total(12.5, 4))
3
```

```
math_utils.py X app.py X test_math_utils.py X
test_math_utils.py > ...
1 from math_utils import calculate_total
2
3 def test_total_price():
4     assert calculate_total(12.5, 4) == 50.0
5
```

```
math_utils.py X app.py test_math_utils.py X
app.py
1 import os
2
3 from math_utils import calculate_total
4 print(calculate_total(12.5, 4))
5
```

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q06$ ruff check . --fix
Found 2 errors (2 fixed, 0 remaining).
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q06$ ruff check .
All checks passed!
```

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q06$ pytest test_math_utils.py -v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/ykw/dev_tools_lab/week2/q06
plugins: typeguard-4.4.2
collected 1 item

test_math_utils.py::test_total_price PASSED

===== 1 passed in 0.02s =====
```

在 VS Code 中借助 Python 语言服务器，成功完成跳转定义与查找所有引用操作，工具找到了

该函数全部三处引用位置。

通过重命名符号语义重构，函数名 `total_price` 被统一修改为 `calculate_total`，三处引用同步更新，没有出现漏改、错改的问题，实现跨文件的安全重构。

手动添加的未使用导入语句 `import os` 被 `ruff` 检测出来，经过自动修复后无用导入已被删除。

执行 `ruff check`，后代码检查无报错警告，运行 `pytest` 测试用例成功通过，证明重构后程序逻辑功能正常。

1.3 q07: 用调试器定位归并排序缺陷

实验内容

```
mkdir -p q07
```

创建实验目录 `q07`

```
cd q07
```

切换工作目录至 `q07`

```
vim merge_sort.py
```

创建带有缺陷的归并排序源代码文件

```
python3 merge_sort.py
```

运行缺陷代码，观察错误输出结果

```
python3 -m pdb merge_sort.py
```

启动 `pdb` 调试器，设置断点，观察变量定位缺陷

`pdb` 调试操作：

1. ``b merge`` 在 `merge` 函数入口打上断点
2. ``c`` 继续运行，触发断点
3. ``p i,j,left[i],right[j]`` 打印变量观察错误
4. ``q`` 退出调试器

```
vim merge_sort.py
```

修改缺陷代码，将 `right[i]` 修正为 `right[j]`

```
vim test_merge.py
```

创建 `pytest` 测试文件，添加两组测试用例

```
pytest test_merge.py -v
```

运行单元测试，验证修复后的程序正确性

实验结果

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q06$ cd ..
ykw@ykw-VirtualBox:~/dev_tools_lab/week2$ mkdir -p q07
ykw@ykw-VirtualBox:~/dev_tools_lab/week2$ cd q07
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ vim merge_sort.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 -m pdb merge_sort.py
> /home/ykw/dev_tools_lab/week2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) █

```

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q06$ cd ..
ykw@ykw-VirtualBox:~/dev_tools_lab/week2$ mkdir -p q07
ykw@ykw-VirtualBox:~/dev_tools_lab/week2$ cd q07
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ vim merge_sort.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 -m pdb merge_sort.py
> /home/ykw/dev_tools_lab/week2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb)

```

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 -m pdb merge_sort.py
> /home/ykw/dev_tools_lab/week2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) --KeyboardInterrupt--
(Pdb) b merge
Breakpoint 1 at /home/ykw/dev_tools_lab/week2/q07/merge_sort.py:2
(Pdb) c
> /home/ykw/dev_tools_lab/week2/q07/merge_sort.py(2)merge()
-> result = []
(Pdb) p i,j,left[i],right[j]
*** NameError: name 'i' is not defined
(Pdb) q
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ vim merge_sort.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ vim test_merge.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ pytest test_merge.py -v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/ykw/dev_tools_lab/week2/q07
plugins: typeguard-4.4.2
collected 2 items

test_merge.py::test_case1 PASSED [ 50%]
test_merge.py::test_case2 PASSED [100%]

===== 2 passed in 0.01s =====
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ ^C
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]

```

运行初始带缺陷的归并代码，输出排序结果错误，证明代码内部存在逻辑缺陷。

借助 pdb 调试器在 merge 函数处设置断点，跟踪观察下标 i,j 的值，定位出错误根源：else 分支读取数组时下标误用 right[i]，正确下标应为 right[j]。

完成最小代码修改，仅修正数组下标，没有替换整体排序算法。修改后运行主程序，数组输出结果符合升序排序预期。

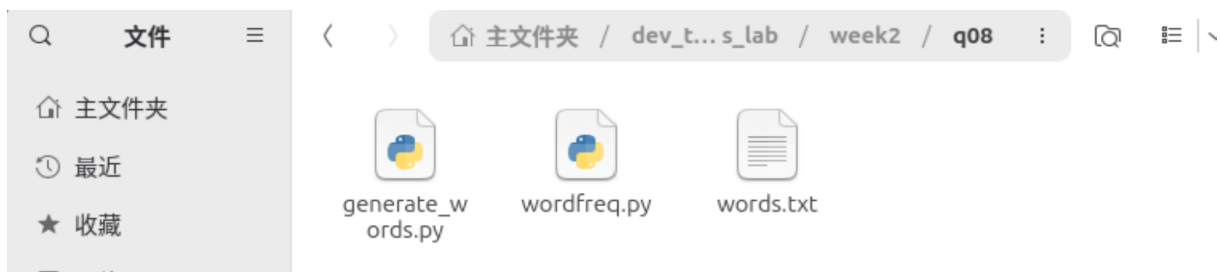
执行 `pytest` 单元测试，原始样例数组、带有重复元素的测试用例全部测试通过。修复后的归并排序程序功能正常，本次实验完成。

1.4 q08: 先测量，再优化慢速词频程序

实验内容

```
mkdir -p q08
创建实验目录 q08
cd q08
切换工作目录至 q08
vim generate_words.py
创建脚本，用于生成words.txt测试数据集
vim wordfreq.py
编写低效的列表去重词频统计原始代码
python3 generate_words.py
生成测试文本文件 words.txt
time python3 wordfreq.py
第一次运行原始程序，记录运行耗时
time python3 wordfreq.py
第二次运行原始程序，记录运行耗时
python3 -m cProfile -s cumulative wordfreq.py
使用cProfile剖析性能，定位程序性能瓶颈
vim wordfreq.py
修改源代码，使用集合set替换列表完成去重优化
time python3 wordfreq.py
第一次运行优化后程序，记录耗时
time python3 wordfreq.py
第二次运行优化后程序，记录耗时
```

实验结果




```
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ python3 generate_words.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.119s
user    0m0.026s
sys     0m0.078s
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.115s
user    0m0.059s
sys     0m0.052s
```

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ vim wordfreq.py
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.023s
user    0m0.007s
sys     0m0.012s
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$ time python3 wordfreq.py
count= 1000

real    0m0.022s
user    0m0.007s
sys     0m0.010s
ykw@ykw-VirtualBox:~/dev_tools_lab/week2/q08$
```

运行生成脚本成功产出 words.txt 测试文件。原始程序使用列表存储唯一单词，通过 `word not in unique` 判断重复，列表成员查询时间复杂度较高，两次运行均消耗较长时间。

经过 cProfile 性能分析，循环内的成员判断占用了绝大部分运行时间，是本次程序的性能瓶颈。

优化时仅将列表替换为集合 `set`，集合查询操作时间复杂度为 $O(1)$ ，并且最终输出的唯一单词数量 `count` 结果保持 1000，未改变程序输出。

优化完成后，两次运行耗时相比原版大幅下降，程序运行速度显著提升。对比优化前后两次耗时的中位数，可计算出本次优化带来的加速比，完成先测量性能、再针对性优化的实验目标。

2 课后

2.1 练习 1

实验问题

解释 Shell 中 stdout 和 stderr 的区别。写出命令，执行 `ls /notexist`，只把错误信息保存到 `error.log` 文件，正常输出直接丢弃。

实验内容

```
ls /notexist 2> error.log > /dev/null
```

执行命令，标准错误输出写入 `error.log`，标准输出丢弃

实验结果

stdout（标准输出）用来输出程序正常运行结果；stderr（标准错误）用来输出报错、警告信息。二者是两条相互独立的输出流，可以分开重定向。执行该命令后屏幕无输出，错误信息保存到文件 `error.log`。

2.2 练习 2

实验问题

在终端执行 `sleep 60`，使用 `Ctrl-Z` 将任务挂起，把任务放到后台继续运行，再使用命令结束该后台进程。写出完整的操作命令序列。

实验内容

```
sleep 60
```

启动休眠任务

```
# 按下快捷键 Ctrl+Z
```

挂起当前前台进程

```
bg
```

将挂起的任务切换至后台继续运行

```
jobs
```

查看后台任务列表

```
kill $(jobs -p)
```

向后台进程发送终止信号

```
wait
```

等待后台进程完全结束

实验结果

`sleep` 进程先被暂停，`bg` 命令恢复后台运行，`kill` 命令成功结束后台休眠进程，任务退出，无残留后台作业。

2.3 练习 3

实验问题

编写一个 `bash` 别名 `lh`，实现等价于 `ls -lh --color=auto` 的效果，要求该别名仅在当前终端会话生效。

实验内容

```
alias lh='ls -lh --color=auto'
```

在当前会话创建 `lh` 别名，关闭终端后别名失效

实验结果

输入 `lh` 即可执行 `ls -lh -color=auto`；别名仅当前终端生效，新开终端别名消失，符合临时会话要求。

2.4 练习 4

实验问题

什么是 Shell 的返回码？Shell 中约定返回码为 0 和非 0 分别代表什么含义？举例说明 `和` `||` 是如何基于返回码工作的。

实验内容

```
mkdir testdir && echo "创建成功"
```

`&&`：前一条成功才执行后一条命令

```
ls nonexistent || echo "文件不存在"
```

`||`：前一条失败才执行后一条命令

实验结果

返回码是程序结束后交给 Shell 的整数状态标记。返回码 0：命令执行成功；非 0：命令执行失败。`&&` 和 `||` 依靠返回码决定是否执行后续命令。

2.5 练习 5

实验问题

使用 Vim（正常模式）操作：打开一个文本文件，给出按键序列：跳到文件第 80 行，删除从前光标到本行末尾全部字符，再复制当前整行。

实验内容

80G

跳转光标至文件第80行

D

删除当前光标位置到本行末尾所有字符

yy

复制当前完整一行到寄存器

实验结果

完成跳转、截断行尾文本、复制整行的操作；全部按键均在 Vim 正常模式执行，无需进入编辑模式。

2.6 练习 6

实验问题

简述什么是语言服务器协议 (LSP)，以 VS-Code + Pylance 为例，列举至少三项语言服务器可以提供的代码功能。

实验结果

语言服务器协议 (LSP) 是编辑器和后端代码分析程序之间的通信标准。将代码分析能力独立为语言服务器，一份服务器可被多款编辑器复用。以 Pylance 为例可提供功能：跳转到定义、查找所有引用、代码重命名重构、语法实时检查、自动代码补全。

2.7 练习 7

实验问题

给定有缺陷的 Python 代码，使用 `python3 -m pdb script.py` 调试，请写出 `pdb` 操作：在函数入口设置断点、继续运行、打印局部变量、退出调试器的对应子命令。

实验内容

```
python3 -m pdb script.py
```

启动pdb调试器加载目标Python脚本

`b` 函数名

在指定函数入口打上断点

`c`

继续运行程序直到触发断点

`p` 变量名

打印查看局部变量的值

`q`

退出pdb调试会话

实验结果

程序在目标函数入口暂停，可查看变量状态；调试结束后安全退出调试环境。

2.8 练习 8

实验问题

Linux 下 `time` 命令输出包含 `real`、`user`、`sys` 三个时间，请分别说明三者代表什么含义。

实验内容

```
time ./demo_program
```

使用time工具测量程序运行时间

实验结果

`real`：实际墙上时钟总耗时，包含休眠、IO 等待时间。`user`：用户态 CPU 耗时，程序自身代码运算消耗的 CPU 时长。`sys`：内核态 CPU 耗时，调用操作系统内核服务消耗的 CPU 时长。

2.9 练习 9

实验问题

简述 `strace` 工具的作用，写出命令跟踪 `cat test.txt` 产生的全部系统调用，并跟随子进程。

实验内容

```
strace -f cat test.txt
```

跟踪cat程序所有系统调用，并且跟随子进程

实验结果

`strace` 工具用来追踪进程发起的操作系统调用，用于排查程序故障。参数 `-f` 的作用为开启跟随子进程，子进程的系统调用同样会被打印输出。

2.10 练习 10

实验问题

有一段运行异常的程序，有两种调试思路：printf 打印调试和使用专用调试器。对比两种方案各自的适用场景。

实验结果

printf 打印调试：适合小型问题快速调试，不需要额外调试工具；缺点是必须修改源代码，调试结束后还要删除打印语句，不能回溯程序执行流程。专用调试器：适合复杂深层缺陷；支持断点暂停、单步运行，无需修改源码；适合定位分支复杂、难以复现的 Bug。

3 附录

github 链接：https://github.com/mimimi67/dev_tools_lab.git

commit 记录截图：

```
ykw@ykw-VirtualBox:~/dev_tools_lab$ git log
commit e8256d4179d7877b2a82e32b990abfe9079bb42b (HEAD -> main, origin/main)
Author: mimimi67 <1873904077@qq.com>
Date: Fri Sep 4 15:42:22 2026 +0800
```

添加week2目录及内部项目文件

```
commit 63b10f7968c04d0d2fd2cc4559a19329b30c81bb
Author: mimimi67 <1873904077@qq.com>
Date: Fri Sep 4 15:42:05 2026 +0800
```

移除残留的week2子模块标记

```
commit d5a3b0b1848cac7249bc8f8d9fc18c990daff5b4
Author: mimimi67 <1873904077@qq.com>
Date: Fri Sep 4 15:39:55 2026 +0800
```

修复：移除week2嵌套的git仓库，整理目录结构

```
commit 99ada0f8fee0d8890350757975283bd79f485973
Author: mimimi67 <1873904077@qq.com>
Date: Mon Aug 31 20:50:01 2026 +0800
```

提交实验报告

```
commit a9b2f39de39df7c9ad316704412d8a67e803def0
Author: mimimi67 <1873904077@qq.com>
Date: Mon Aug 31 20:15:37 2026 +0800
```

提交课后练习